

第14周实验指导书：Git分支合并与多用户架构改造

实验基本信息

课程名称：高级程序设计与系统开发工具

实验章节：第14周 服务层设计与业务逻辑（综合实践）

实验学时：4学时

实验类型：综合型实验

实验分组：个人独立完成

前置知识：Git基础操作、Spring Data JPA、Spring Boot Web开发

实验目的与要求

1. 实验目的

- 掌握Git分支管理策略**：学会处理多分支开发后的合并（Merge）、冲突解决以及版本标签（Tag）管理。
- 理解多用户系统架构**：从单用户系统向多用户系统演进，理解数据隔离的基本原理。
- 掌握ThreadLocal的使用**：学习如何使用 `ThreadLocal` 在同一线程内传递用户上下文信息，避免在每个方法参数中显式传递用户信息。
- 服务层重构能力**：在不改变前端接口契约的前提下，对后端服务层和持久层进行重构。

2. 实验要求

- 完成代码合并**：将之前分散在 `feature/xxx` 等分支的代码合并到 `master` 主分支，并打上 `v1.0-single-user` 标签。
- 数据模型升级**：修改核心实体类，增加用户归属字段。
- 实现模拟登录**：在不引入 Spring Security 等复杂安全框架的前提下，通过拦截器实现“默认用户”登录，模拟多用户环境。
- 实现数据隔离**：确保 Service 层在查询和保存数据时，自动关联当前登录用户。

实验环境与工具

- JDK: 17+
- IDE: IntelliJ IDEA / Eclipse / VS Code
- **构建工具**: Maven
- **版本控制**: Git
- **数据库**: H2 / MySQL

实验内容与步骤

第一部分: Git 分支管理与合并 (1学时)

在进行多用户改造之前, 我们需要先将之前的开发成果进行整合, 确立一个稳定的单用户版本基线。

任务 1-1: 查看与整理分支

1. 打开终端或 Git 客户端, 查看当前所有分支:

```
git branch -a
```

2. 确保你已经提交了当前工作区的所有修改。

任务 1-2: 合并分支到 Master

假设你之前有 `feature/jpa` (持久层改造)、`feature/aop` (切面编程) 等分支。我们需要将它们合并到 `master`。

1. 切换到 `master` 分支:

```
git checkout master
```

2. 执行合并操作 (以合并 `feature/aop` 为例) :

```
git merge feature/aop
```

3. **处理冲突**: 如果出现冲突 (Conflict), 打开冲突文件, 手动选择保留的代码, 然后执行 `git add` 和 `git commit` 完成合并。

任务 1-3: 发布 v2.0 单用户版本

合并完成后，确保项目可以正常运行。此时的项目是一个功能完整的**单用户**个人知识管理系统。我们给它打上标签，作为里程碑。

```
git tag -a week14-v2.0-single-user -m "Release version 2.0: Single user PKM system"
git push origin master --tags
```

第二部分：多用户数据模型改造（1学时）

在进行代码修改之前，我们需要先理解单用户系统与多用户系统在数据层面的本质区别。

2.1 核心概念：数据隔离与归属

- **单用户模式：**

- **场景：**软件运行在个人电脑上，只有拥有者一个人使用。
- **数据特征：**数据库中的所有数据默认都属于当前用户。
- **查询逻辑：** `SELECT * FROM note` —— 拿走所有笔记。

- **多用户模式：**

- **场景：**软件部署在云端，成百上千的用户通过浏览器同时访问。
- **数据特征：**所有用户的笔记都存储在同一张 `note` 表中。
- **查询逻辑：** `SELECT * FROM note WHERE user_id = ?` —— **必须**区分数据是谁的。如果不加过滤，张三就能看到李四的私密日记，这是严重的系统漏洞。

因此，改造的第一步，就是给所有的业务数据（笔记、标签、分类）贴上“标签”，指明它们的主人是谁。

任务 2-1：创建 User 实体（如果尚未存在）

如果项目中还没有 `User` 实体，请创建一个简单的 `User` 实体。

```

@Entity
@Table(name = "users")
@Data
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true, nullable = false)
    private String username;

    private String password; // 暂时存储明文或简单加密
    private String email;
}

```

任务 2-2：改造核心业务实体

修改 `Note` (笔记)、`Category` (分类)、`Tag` (标签) 等实体，增加 `userId` 字段。

方案 A：直接添加 `userId` 字段（简单，性能好，推荐本次实验使用） 方案 B：添加 `User` 对象关联（`@ManyToOne`，符合面向对象，但配置稍繁琐）

我们采用**方案 A**，在 `Note` 等实体中添加：

```

@Entity
public class Note {
    // ... 原有字段 ...

    @Column(name = "user_id", nullable = false)
    private Long userId;

    // ...
}

```

注意：

1. 对于现有的数据，添加 `nullable = false` 可能会导致启动报错。可以先设置为 `nullable = true`，或者在数据库中手动更新旧数据的 `user_id` 为默认值（如 1），然后再改为 `false`。
2. 或者配置 `spring.jpa.hibernate.ddl-auto=update` 让 Hibernate 尝试更新结构（可能需要手动干预）。

第三部分：服务层改造与模拟上下文（1.5学时）

这是本次实验的核心。我们需要在不改变 Controller 接口参数的情况下，让 Service 层知道“当前是谁在操作”。

任务 3-1: 创建用户上下文工具类 (UserContext)

利用 `ThreadLocal` 存储当前线程的用户 ID。

```
public class UserContext {  
    private static final ThreadLocal<Long> CURRENT_USER = new ThreadLocal<>();  
  
    public static void setCurrentUserId(Long userId) {  
        CURRENT_USER.set(userId);  
    }  
  
    public static Long getCurrentUserId() {  
        return CURRENT_USER.get();  
    }  
  
    public static void clear() {  
        CURRENT_USER.remove();  
    }  
}
```

任务 3-2: 实现模拟登录拦截器

创建一个拦截器，在请求进入 Controller 之前，将默认用户 ID 放入 `UserContext`。

1. 创建拦截器 `UserInterceptor`：

```

@Component
public class UserInterceptor implements HandlerInterceptor {
    @Override
    public boolean preHandle(HttpServletRequest request, HttpServletResponse response,
Object handler) {
        // 模拟：假设当前登录用户是 ID 为 1 的用户
        // 在后续完善中，这里会从 Session 或 JWT Token 中解析用户 ID
        Long defaultUserId = 1L;
        UserContext.setCurrentUserId(defaultUserId);
        return true;
    }

    @Override
    public void afterCompletion(HttpServletRequest request, HttpServletResponse
response, Object handler, Exception ex) {
        // 请求结束后清理 ThreadLocal，防止内存泄漏
        UserContext.clear();
    }
}

```

2. 注册拦截器：

```

@Configuration
public class WebConfig implements WebMvcConfigurer {
    @Autowired
    private UserInterceptor userInterceptor;

    @Override
    public void addInterceptors(InterceptorRegistry registry) {
        registry.addInterceptor(userInterceptor)
            .addPathPatterns("/api/**"); // 拦截所有 API 请求
    }
}

```

任务 3-3：改造 Repository 和 Service

确保用户只能操作自己的数据。

1. Repository 层：在 `NoteRepository` 中增加带 `userId` 的查询方法。

```
public interface NoteRepository extends JpaRepository<Note, Long> {
    // 原有: List<Note> findAll();
    // 新增:
    List<Note> findByUserId(Long userId);

    // 原有: Optional<Note> findById(Long id);
    // 新增: 确保只能查到自己的笔记
    Optional<Note> findByIdAndUserId(Long id, Long userId);
}
```

2. **Service 层**: 修改 `NoteService` , 在所有业务逻辑中注入 `userId` 。

```
@Service
public class NoteService {
    @Autowired
    private NoteRepository noteRepository;

    public List<NoteDTO> getAllNotes() {
        Long currentUserId = UserContext.getCurrentUserId();
        // 只查询当前用户的笔记
        List<Note> notes = noteRepository.findByUserId(currentUserId);
        return
        notes.stream().map(this::convertToDTO).collect(Collectors.toList());
    }

    public NoteDTO createNote(NoteDTO noteDTO) {
        Long currentUserId = UserContext.getCurrentUserId();
        Note note = convertToEntity(noteDTO);

        // 强制设置归属用户
        note.setUserId(currentUserId);

        Note saved = noteRepository.save(note);
        return convertToDTO(saved);
    }

    // 其他方法同理改造...
}
```

第四部分：验证与测试 (0.5学时)

任务 4-1: 验证默认用户功能

1. 启动项目。

2. 使用 Postman 或前端页面访问笔记列表接口。
3. 观察数据库，确认新插入的笔记 `user_id` 字段是否自动填充为 `1`。

任务 4-2：多用户隔离测试（模拟）

1. 修改 `UserInterceptor`，将默认用户 ID 改为 `2L`。
2. 重启项目。
3. 再次访问笔记列表接口。
 - **预期结果**：应该看不到之前用户 1 创建的笔记（列表为空，或只显示用户 2 的数据）。
4. 创建一条新笔记。
5. 观察数据库，确认新笔记的 `user_id` 为 `2`。

提交

```
git commit
git tag -a week14-multi-user -m "v3.01 multi-user system"
git push origin master --tags
```

实验总结

通过本次实验，我们完成了从单用户应用到多用户应用的架构跨越。虽然我们暂时没有实现复杂的登录页面和密码验证，但通过 `ThreadLocal` 和 **拦截器** 机制，我们在后端核心业务层建立了牢固的“用户上下文”概念。这是后续接入 Spring Security 或 JWT 认证的基础。